Métricas de avaliação de texto gerado por LLM

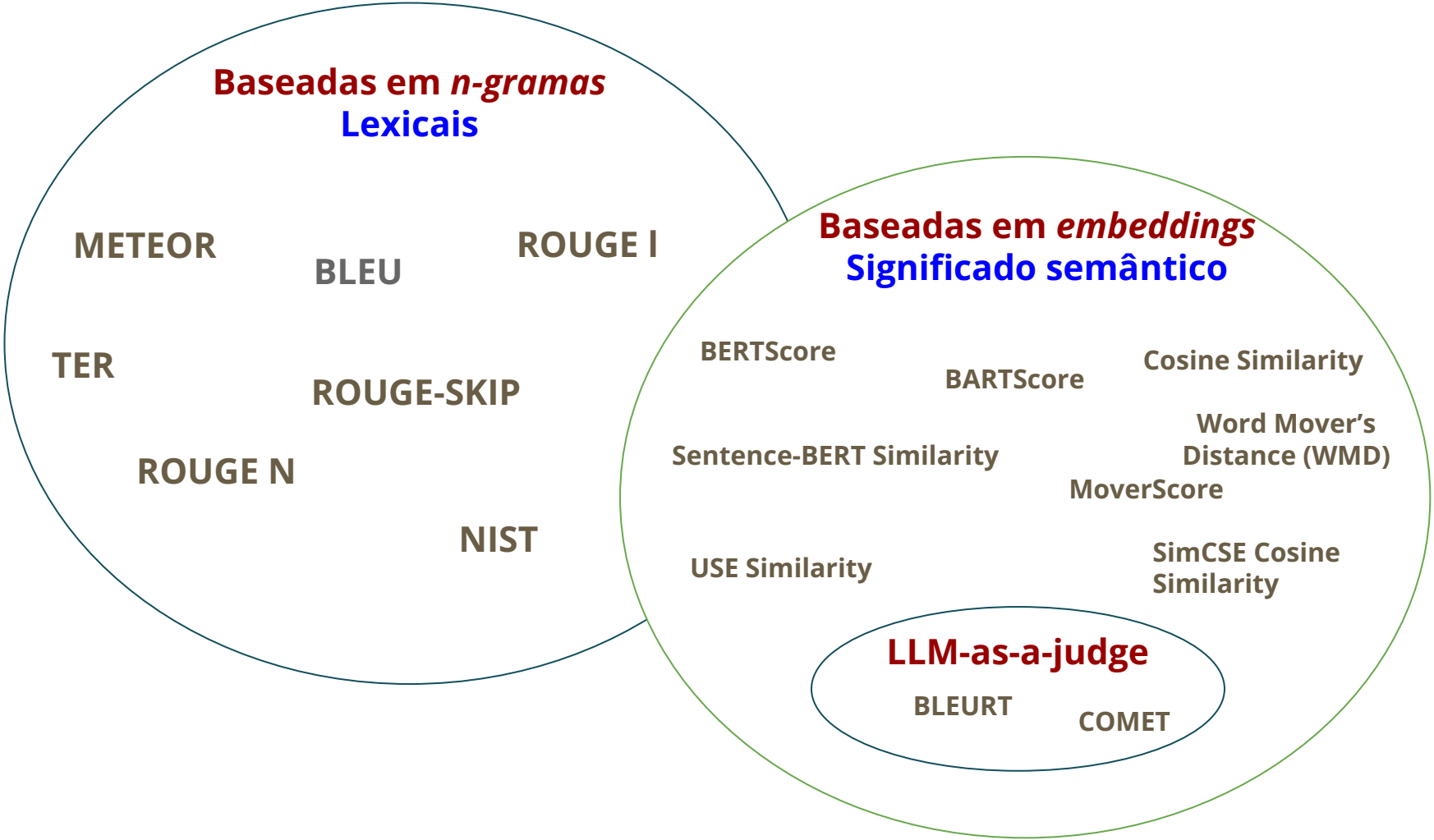
Em comparação a textos de referência humana

Métricas de avaliação

Todas as métricas que serão apresentadas são métricas comparativas entre resposta da LLM e resposta humana

Exemplo que servirá como base

- **humano = “O gato sentou no tapete.”**
- **llm = “O gato preto e fofinho sentou-se confortavelmente sobre o tapete.”**


Baseadas em *n-gramas*
Lexicais
METEOR
BLEU
ROUGE l
TER
ROUGE-SKIP
ROUGE N
NIST
Baseadas em *embeddings*
Significado semântico
BERTScore
BARTScore
Cosine Similarity
Word Mover's Distance (WMD)
Sentence-BERT Similarity
MoverScore
USE Similarity
SimCSE Cosine Similarity
LLM-as-a-judge
BLEURT
COMET

BLEU

BLEU (*Bilingual Evaluation Understudy*) é um algoritmo de avaliação de texto traduzido por máquina de uma língua para outra

$$\text{BLEU} = BP \cdot \exp\left(\frac{1}{N}\sum_{n=1}^{N} \log p_n\right)$$

BP = penalidade quando a resposta é <u>menor</u> do que a referência

pn = quantifica a precisão dos *n-gramas* avaliando a proporção de *n-gramas* compartilhados entre a tradução gerada por máquina e o texto de referência, em comparação com a contagem geral de *n-gramas* na tradução gerada por máquina

Precisão = número de palavras iguais / número total de palavras (**LLM**)

exp = valor de e (2,718...) elevado a 1/n....

Valor mínimo = 0, **valor máximo=** 1. Quanto **maior**, melhor. *O(n)*

BLEU

humano = “O gato sentou no tapete.”

llm = “O gato preto e fofinho sentou-se confortavelmente sobre o tapete.”

Primeiro, contamos quantas palavras da frase llm aparecem na frase de humana (unigramas)

qtde_humano = 5

qtde_llm = 10

Precisão = 3/10 = 0.3 - para unigramas *(o, gato, tapete)*

O cálculo da precisão é repetido para bi, tri e quadri gramas

Podem ser calculado para menos n-gramas ou até mais do que 4, mas recomenda-se 4

BLEU

$$\text{BLEU-4} = 1 \cdot \exp\left(\frac{1}{4}(\log 0.3 + \log 0.1111 + \cancel{\log 0} + \cancel{\log 0})\right) = 0$$

Não houve correspondência exata dos *n-gramas* avaliados (4 no caso)

BP= 1 porque a resposta da **llm** foi maior que a **humana**, nesse caso, não se calcula nenhuma penalidade

¼ = porque são usados 4 *n-gramas*

Soma dos logaritmos das precisões, no caso, para tri e quadri gramas é 0

BLEU para o nosso caso

Poderíamos usar a BLEU para uma medição rápida da sobreposição exata de termos

Porém, ela é MUITO radical no geral (pode ser adaptada), mas se não tiver trigramas ou quadrigramas, como no exemplo, e o fato dela trabalhar com *log*, já faz com que o resultado seja 0

A desvantagem é que ele **não entende sinônimos** e, como a resposta do LLM é sempre mais longa, a nota de precisão será normalmente baixa

NIST

O nome vem de "*National Institute Standard Technology*", é baseada na BLEU, porém, prioriza palavras mais importantes **(raras)**, descartando menos importantes **(muito frequentes)**

humano = "O gato sentou no tapete."

llm = "O gato preto e fofinho sentou-se confortavelmente sobre o tapete."

qtde_humano = 5

qtde_llm = 10

$$\text{NIST} = \text{Penalidade de comprimento} \times \sum_{n=1}^{N} \sum_{\text{todos os n-grams correspondentes}} \text{info}(n\text{-gram})$$

NIST

$$\text{NIST} = (2.64 + 0.32 + 0) \times 1 = 2.96$$

Onde:

2.64 - soma das informações dos unigramas correspondentes ("o", "gato", "tapete")

0.32 - soma das informações dos bigramas correspondentes ("o gato")

0 - nenhum trigrama ou quadrigrama

O NIST usa como corpus as referências humanas que são repassadas para análise

$$\text{info}(n\text{-gram}) = \log_2 \frac{\text{total de n-grams do mesmo tamanho no corpus}}{\text{ocorrências do n-gram no corpus}}$$

1 - penalidade, segue a mesma lógica do BLEU, sem penalidade no nosso exemplo

Valor mínimo = 0, **valor máximo=** sem valor definido. Quanto **maior**, melhor. *O(n)*

NIST - para o nosso caso

Dá mais importância para termos técnicos (palavras raras) que coincidam. Mas também não entende sinônimos

TER

TER (T*ranslation Edit Rate*) mede quanto esforço de edição seria necessário para transformar a tradução candidata (gerada por um sistema) em uma tradução de referência humana

Quanto mais próximo de 0, melhor

Edição pode ser = inserção, exclusão e substituição de palavras isoladas, bem como deslocamentos de sequências de palavras

$$\text{TER} = \frac{\text{\# of edits}}{\text{average \# of reference words}}$$

Valor mínimo = 0, **valor máximo=** sem limite definido. Quanto **menor**, melhor. *O(n*m)*

TER

humano = “O gato sentou no tapete.”

llm = “O gato preto e fofinho sentou-se confortavelmente sobre o tapete.”

1. Manter `"o"` (já igual).
2. Manter `"gato"`.
3. **Deletar** `"preto"`. (1)
4. **Deletar** `"e"`. (2)
5. **Deletar** `"fofinho"`. (3)
6. **Substituir** `"sentou-se"` → `"sentou"`. (4)
7. **Deletar** `"confortavelmente"`. (5)
8. **Substituir** `"sobre"` → `"no"`. (6)
9. **Deletar** o token extra `"o"` antes de `tapete`. (7)
10. Manter `"tapete"`.

Total de edições = **7**.

TER = 7/5 = **1.4**

Isso significa que, em média, seriam necessárias 1,4 edições para cada palavra da referência para corrigir a candidata
Ou seja: <u>tradução ruim</u>, já que o valor é bem maior que 1

Somente 1 resposta de referência? Usar a quantidade de palavras dela, não será calculada uma média

TER - Algoritmo de cálculo do número de edições

Algorithm 1 Calculate Number of Edits

```
input: HYPOTHESIS h
input: REFERENCES R
E ← ∞
for all r ∈ R do
    h' ← h
    e ← 0
    repeat
        Find shift, s, that most reduces min-edit-
        distance(h', r)
        if s reduces edit distance then
            h' ← apply s to h'
            e ← e + 1
        end if
    until No shifts that reduce edit distance remain
    e ← e+ min-edit-distance(h', r)
    if e < E then
        E ← e
    end if
end for
return E
```

TER - para o nosso caso

A desvantagem do TER é que ele contaria todas as palavras extras do LLM como "erros" (deleções), resultando em uma nota muito ruim apenas por causa da resposta ser maior

ROUGE N

A métrica ROUGE N calcula a sobreposição de *n-gramas* que aparecem na frase gerada por LLM, que estão presentes na frase de referência

humano = “O gato sentou no tapete.”

llm = “O gato preto e fofinho sentou-se confortavelmente sobre o tapete.”

unigrama = “o”, “gato”, “sentou”, “no”, “tapete”

bigrama = “o gato”, “gato sentou”, “no tapete”

trigrama, quadrigrama, etc... podem ser avaliados

Valor mínimo = 0, **valor máximo=** 1. Quanto **maior**, melhor. *O(n)*

ROUGE N

Precisão (P) = *n-gramas* identificados / n-gramas da resposta gerada por LLM

Revocação (recall) (R) = *n-gramas* identificados / n-gramas da referência

F1-Score = (2 * P * R) / (P + R)

Para todas as versões de ROUGE, são esses os cálculos realizados

ROUGE L e ROUGE S

A ROUGE-L é baseada em LCS (*Longest Common Subsequence*) - maior subsequência comum entre o texto gerado e o texto de referência - não precisam ser consecutivas, desde que respeitem a ordem de aparição

A ROUGE-S é baseada em *skip-bigramas*, ou seja, todos os pares de palavras possíveis que seguem a mesma ordem, não necessitando serem consecutivos, ex.: {o gato}, {o dorme}, {o tapete}, {o no}, {gato dorme}, {gato tapete}, {gato no}....

ROUGE - para o nosso caso

Seria útil num contexto de cobertura (*recall*), para respostas que precisam ter aquelas “x” palavras, mas não entende sinônimos e acaba penalizando respostas de tamanhos maiores

Já temos os resultados da ROUGE - 1 e da ROUGE - L

METEOR

A METEOR (*Metric for Evaluation of Translation with Explicit ORdering*) é uma métrica de avaliação automática que tenta capturar similaridade semântica e de ordem, indo além da contagem simples de *n-gramas*, como ROUGE ou BLEU

- Alinhamento palavra a palavra entre referência e candidato
- Sinônimos e *stemming* (flexão de palavras) - dicionário *Wordnet*
- Precisão (P) e Revocação (R)
- Penalidade por fragmentação (quão desordenadas estão as correspondências)

$$\text{METEOR} = F_{\text{mean}} \cdot (1 - \text{penalty})$$

Valor mínimo = 0, **valor máximo=** 1. Quanto **maior**, melhor. *O(n*m)*

METEOR - Passo 1 - Calcula-se P (precisão) e Recall (R)

humano = **“O gato sentou no tapete.”**

llm = **“O gato preto e fofinho sentou-se confortavelmente sobre o tapete.”**

Alinhamentos possíveis:

Referência	Candidato	Tipo de correspondência
O	O	Igual
gato	gato	Igual
sentou	sentou-se	Stemming
no	sobre	–
tapete	tapete	Igual

- Palavras extras do candidato: "preto", "e", "fofinho", "confortavelmente" → não alinhadas.

$$P = \frac{4}{10} = 0,4$$

$$R = \frac{4}{5} = 0,8$$

METEOR - Passo 2 - Calcula-se F1 harmônico

$$F1 = \frac{2 \cdot P \cdot R}{P + R} = \frac{2 \cdot 0,4 \cdot 0,8}{0,4 + 0,8} \approx 0,533$$

METEOR - Passo 3 - Aplicar penalidade (se existir)

Caso alguma palavra alinhada (que exista em ambos os textos) esteja em ordem diferente no texto gerado, uma penalidade deve ser aplicada, seguindo a fórmula $Pen = \gamma \cdot frag^{\beta}$

Um *chunk* é uma sequência contígua de palavras alinhadas entre a resposta gerada e a referência que mantém a mesma ordem

Sendo:

Y = parâmetro de escala de penalidade, normalmente ≈ 0.5

frag = fragmentação $\text{frag} = \frac{\text{número de chunks}}{\text{número de palavras alinhadas}}$

β = parâmetro que controla a sensibilidade da penalidade à fragmentação (padrão ≈ 3)

penalty=0.5 · (1/4)^3=0.5 · 0.015625≈0.0078

METEOR - Resultado final

$$\text{METEOR} = F_{\text{mean}} \cdot (1 - \text{penalty})$$

$$\text{METEOR} \approx 0.727 \cdot (1 - 0.0078) \approx 0.727 \cdot 0.9922 \approx 0.721$$

METEOR - Para o nosso caso

Consegue identificar sinônimos e radicais (dadas as devidas proporções) avaliando melhor as paráfrases do LLM

A desvantagem é que esse conhecimento semântico é limitado a um dicionário específico e não possui o entendimento contextual mais profundo

Palavras não alinhadas prejudicam precisão e *recall*

Cosine Similarity

A Similaridade de Cosseno é uma métrica usada para medir a similaridade entre dois vetores, baseando-se no ângulo e não na magnitude

Valores de **-1 a 1**, sendo:

- 1: vetores apontam na mesma direção → textos muito semelhantes
- 0: vetores ortogonais → textos sem relação
- -1: vetores opostos → textos com sentido oposto

$$\text{sim}(A, B) = \cos(\theta) = \frac{A \cdot B}{\|A\|\|B\|}$$

Valor mínimo = -1, **valor máximo=** 1. Quanto **maior**, melhor. $O(n^2)$

Cosine Similarity - Representação como vetor

- O algoritmo converte cada texto em um vetor numérico
- Pode ser *Bag-of-Words* (contagem de palavras), *TF-IDF* (ponderação por frequência/inverso da frequência) ou *embeddings* (vetores densos que capturam significado)
  - São formas de representação vetorial
  - *Embeddings* são mais usados pois capturam mais contexto semântico

- **Referência (humano):**

$$A = [1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0]$$

- **Candidata (LLM):**

$$B = [2, 1, 0, 0, 1, 1, 1, 1, 1, 1, 1]$$

Cosine Similarity - Produto interno

- O algoritmo multiplica cada componente correspondente dos dois vetores e soma os resultados
- Esse valor indica quanto os vetores “apontam na mesma direção”

$$A \cdot B = (1 * 2) + (1 * 1) + (1 * 0) + (1 * 0) + (1 * 1) + 0 + ... + 0 = 2 + 1 + 0 + 0 + 1 = 4$$

Cosine Similarity - Magnitude (tamanho) de cada vetor

- Calcula a magnitude de cada vetor : $\|A\| = \sqrt{\sum_i A_i^2}$
- Serve para normalizar os vetores e garantir que o resultado dependa da direção, não do comprimento

$$\|A\| = \sqrt{1^2 + 1^2 + 1^2 + 1^2 + 1^2 + 0 + ... + 0} = \sqrt{5} \approx 2.236$$

$$\|B\| = \sqrt{2^2 + 1^2 + 0^2 + 0^2 + 1^2 + 1^2 + 1^2 + 1^2 + 1^2 + 1^2 + 1^2} = \sqrt{4 + 1 + 0 + 0 + 1 + 1 + 1 + 1 + 1 + 1 + 1} = \sqrt{11} \approx 3.317$$

$$\text{cosine-similarity} = \frac{A \cdot B}{\|A\|\|B\|} = \frac{4}{2.236 * 3.317} \approx \frac{4}{7.414} \approx \boxed{0.54}$$

Sentence-BERT Similarity

A Sentence-BERT Similarity é uma métrica que mede o quão semanticamente próximas duas frases estão usando *embeddings* gerados por um modelo chamado Sentence-BERT (SBERT)

$$\text{cos_sim} = \frac{\vec{v}_1 \cdot \vec{v}_2}{||\vec{v}_1||\,||\vec{v}_2||}$$

É a mesma lógica da Similaridade de Cosseno, mas com o modelo Sentence-BERT como gerador dos *embbedings*

Valor mínimo = 0, **valor máximo=** 1. Quanto **maior**, melhor. $O(n^2)$

Universal Sentence Encoder (USE) Similarity

O *Universal Sentence Encoder* (USE) é um modelo de *embeddings* de sentenças criado pelo Google, que transforma sentenças em vetores densos de dimensão fixa, geralmente com 512 dimensões

A ideia é capturar a semântica das sentenças

Para comparar sentenças, usa-se a similaridade do cosseno entre os vetores gerados

Valor mínimo = 0, **valor máximo=** 1. Quanto **maior**, melhor. $O(n^2)$

SimCSE Cosine Similarity

SimCSE (*Simple Contrastive Sentence Embeddings*) é um modelo de *embeddings* de sentenças baseado em BERT ou RoBERTa. O objetivo é gerar vetores de sentenças mais consistentes semanticamente, de forma que frases com o mesmo significado fiquem mais próximas no espaço vetorial, enquanto frases diferentes fiquem distantes

Valor mínimo = 0, **valor máximo=** 1. Quanto **maior**, melhor. $O(n^2)$

SimCSE Cosine Similarity

É a mesma lógica da Similaridade de Cosseno, porém, o modelo é mais sofisticado, porque ele treina os *embeddings* de forma contrastiva, explicitamente para que frases com o mesmo significado fiquem próximas no espaço vetorial

BERTScore

O BERTScore é uma métrica de avaliação de similaridade de texto que usa modelos de linguagem contextualizados (como BERT) para comparar semântica das palavras, ao invés de apenas palavras exatas como ROUGE ou BLEU

Valor mínimo = 0, **valor máximo=** 1. Quanto **maior**, melhor. $O(n^2)$

BERTScore - Primeiro passo

humano: ["O", "gato", "sentou", "no", "tapete"]

llm: ["O", "gato", "preto", "e", "fofinho", "sentou-se", "confortavelmente", "sobre", "o", "tapete"]

Token humano	Token LLM mais parecido	Similaridade (0-1)
O	O	1.0
gato	gato	1.0
sentou	sentou-se	0.8
no	sobre	0.5
tapete	tapete	1.0

BERTScore - Cálculo da precisão, *recall* e F1-Score

$$Recall = \frac{1.0 + 1.0 + 0.8 + 0.5 + 1.0}{5} = \frac{4.3}{5} = 0.86$$

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall} = 2 \cdot \frac{0.53 \cdot 0.86}{0.53 + 0.86}$$

$$F1 = 2 \cdot \frac{0.4558}{1.39} = 2 \cdot 0.3277 \approx 0.655$$

Token LLM	Token humano mais parecido	Similaridade
O	O	1.0
gato	gato	1.0
preto	-	0.0
e	-	0.0
fofinho	-	0.0
sentou-se	sentou	0.8
confortavelmente	-	0.0
sobre	no	0.5
o	O	1.0
tapete	tapete	1.0

$$Precision = \frac{1.0 + 1.0 + 0.0 + 0.0 + 0.0 + 0.8 + 0.0 + 0.5 + 1.0 + 1.0}{10} = \frac{5.3}{10} = 0.53$$

BERT Score - para o nosso caso

O BERTScore nos daria uma avaliação semântica mais robusta

A desvantagem é a necessidade de interpretar seus *scores* de **Precisão** e ***Recall***

- provavelmente teremos um ***Recall*** alto, mas uma **Precisão** baixa devido à respostas maiores do LLM

BARTScore

O BARTScore é uma métrica de avaliação baseada em modelos de linguagem de *seq2seq* (neste caso, o BART)

A ideia é medir quão provável uma sentença é dada a outra, segundo o modelo BART

- O BART é usado como um gerador: ele recebe a referência e tenta gerar a saída e pode-se usar uma versão bidirecional

$$\text{BARTScore}(x \rightarrow y) = \frac{1}{N} \sum_{i=1}^{N} \log P(y_i \mid y_{<i}, x)$$

Valor mínimo = -infinito, **valor máximo=** 0. Quanto **mais perto de 0**, melhor. $O(n^2)$

BARTScore - Bidirecional

- Pode ser utilizada a versão bidirecional, onde é feita a média entre quão provável é o texto do modelo ser gerado a partir da referência e mede quão provável é o texto da referência ser gerado a partir do modelo

$$\text{BARTScore}(x \rightarrow y) = \frac{1}{N} \sum_{i=1}^{N} \log P(y_i \mid y_{<i}, x)$$

BARTScore - Passo 1

Calcula a probabilidade do modelo gerar cada *token* e considera seus *logs* naturais

```
1. "O"                → P = 0.90
2. "gato"             → P = 0.92
3. "preto"            → P = 0.40
4. "e"                → P = 0.70
5. "fofinho"          → P = 0.30
6. "sentou-se"        → P = 0.60
7. "confortavelmente"→ P = 0.45
8. "sobre"            → P = 0.60
9. "o"                → P = 0.85
10. "tapete"          → P = 0.88
```

```
1. ln(0.90)  = -0.10536
2. ln(0.92)  = -0.08338
3. ln(0.40)  = -0.91629
4. ln(0.70)  = -0.35667
5. ln(0.30)  = -1.20397
6. ln(0.60)  = -0.51083
7. ln(0.45)  = -0.79851
8. ln(0.60)  = -0.51083
9. ln(0.85)  = -0.16252
10.ln(0.88)  = -0.12783
```

Soma todos os valores e divide pelo número de tokens (10)

$$\frac{-4.77619}{10} = -0.47762$$

BARTScore - Passo 2

Calcula a probabilidade do modelo gerar cada *token* e considera seus *logs* naturais

```
1. "O"      → P = 0.90
2. "gato"   → P = 0.92
3. "sentou" → P = 0.65
4. "no"     → P = 0.50
5. "tapete" → P = 0.88
```

```
1. ln(0.90) = -0.10536
2. ln(0.92) = -0.08338
3. ln(0.65) = -0.43078
4. ln(0.50) = -0.69315
5. ln(0.88) = -0.12783
```

Soma todos os valores e divide pelo número de tokens (5)

$$\frac{-1.44051}{5} = -0.28810$$

BARTScore - Passo 3

Cálculo final do BARTScore bidirecional

$$\text{BARTScore}_{bi} = \frac{1}{2}\big(-0.47762 + (-0.28810)\big) = \frac{-0.76572}{2} = -0.38286$$

- $\text{BARTScore}(x \to y) = -0.47762$

  → em média os tokens de y têm essa log-probabilidade condicionada em x.
- $\text{BARTScore}(y \to x) = -0.28810$

  → referência x é relativamente provável dado y (mais próximo de 0 do que a outra direção).
- $\text{BARTScore}_{bi} = -0.38286$

  → score combinado; quanto **mais próximo de 0**, melhor.

BART Score - para o nosso caso

A desvantagem é a interpretabilidade: os *scores* são log-probabilidades (valores negativos como -0.38), tornando difícil criar um senso intuitivo do que é "bom" ou "ruim"

Word Mover's Distance (WMD)

A *Word Mover's Distance* (WMD) é uma métrica de similaridade textual baseada em *embeddings* de palavras. A distância **WMD** mede a dissimilaridade entre dois documentos de texto como a distância mínima que as palavras inseridas em um documento precisam "percorrer" para alcançar as palavras inseridas em outro documento

Valor mínimo = 0, **valor máximo=** sem limite definido. Quanto **menor**, melhor.
$O(p^3 \log p)$, onde p é o número de palavras únicas nos documentos

Word Mover's Distance (WMD) - Passo 1 - Vetorização

O primeiro passo é vetorizar as frases usando modelos de *Word Embeddings* como Word2Vec ou o GloVe

humano = "O gato sentou no tapete."

llm = "O gato preto e fofinho sentou-se confortavelmente sobre o tapete."

Word Mover's Distance (WMD) - Passo 2 - Calcular custo

O segundo passo é calcular os custos de transporte de cada palavra diferente da resposta gerada para cada palavra da resposta de referência

$$\sum_{i=1}^{n} \mathbf{T}_{ij} = d'_j \quad \forall j \in \{1, \ldots, n\}.$$

De (LLM)	Para (Humano)	Vetor (LLM)	Vetor (Humano)	Distância (Custo)
gato	gato	[2, 5]	[2, 5]	0
preto	gato	[3, 8]	[2, 5]	3.16
fofinho	gato	[4, 6]	[2, 5]	2.23
sentou-se	sentou	[4.1, 2.1]	[4, 2]	0.14
...	...	...	...	...

Word Mover's Distance (WMD) - Passo 3 - Menor custo

O terceiro passo é encontrar a maneira mais "barata" de mover o "peso" de todas as palavras da frase do **LLM** para as palavras da frase **humana**. A ordem das palavras não importa.Esse é um processo de otimização caro

- Mover "gato" (LLM) para "gato" (Humano) tem custo 0.
- Mover "sentou-se" (LLM) para "sentou" (Humano) tem um custo muito baixo.
- Mover "tapete" (LLM) para "tapete" (Humano) tem custo 0.
- As palavras extras do LLM ("preto", "fofinho", "confortavelmente", "sobre") precisam ser "transportadas" para as palavras semanticamente mais próximas na frase humana. Por exemplo, "fofinho" e "preto" provavelmente seriam "movidas" para "gato", e "confortavelmente" para "sentou".

O valor final da WMD é o resultado de um processo de otimização complexo que encontra o menor custo total possível

Word Mover's Distance = 1.39 (no nosso exemplo)

Mover Score

A Mover Score, é uma combinação de

- representações contextualizadas de textos do sistema e de referência
- uma distância entre essas representações, medindo a distância semântica entre as saídas do sistema e as referências

É importante para uma métrica refletir com precisão até que ponto o texto do sistema se desviou da referência

Valor mínimo = 0, **valor máximo=** 1. Quanto **maior**, melhor. $O(n^2 + p^3 \log p)$ - onde n= número total de palavras na frase, p=número de palavras únicas (BERT + Mover Score)

Mover Score - Passo 1 - Gerar *embbedings*

O Mover Score usa modelos como BERT, capazes de gerar *embbedings* semânticos e as *stopwords* são removidas

Frase Humana (H):

- gato: `[2.1, 5.2]`
- sentou: `[4.0, 2.0]`
- tapete: `[6.0, 4.1]`

Frase do LLM (L):

- gato: `[2.2, 5.3]` (ligeiramente diferente por causa do "preto e fofinho")
- preto: `[1.5, 6.0]`
- fofinho: `[2.8, 6.1]`
- sentou-se: `[4.05, 2.05]` (contextualizado por "confortavelmente")
- confortavelmente: `[4.5, 1.5]`
- tapete: `[6.1, 4.2]` (contextualizado por "sobre")

Mover Score - Passo 2 e 3 - Atribuir pesos e Calcular custo

O MoverScore reconhece que nem todas as palavras são igualmente importantes. Ele usa o IDF (*Inverse Document Frequency*) para dar um "peso" a cada palavra. Palavras raras e informativas têm um peso maior

- A etapa de cálculo de custos é idêntica ao WMD, porém o transporte ótimo é ponderado por esses pesos IDF e, no final, o custo total é normalizado pela soma dos pesos

Mover Score - Passo 4 - Encontrar menor custo

Agora, o algoritmo de otimização encontra o plano de transporte de menor custo total. Ele vai mover o peso IDF de cada palavra do LLM para as palavras da frase Humana. A solução ótima provavelmente seria:

- Mover o peso de `gato(L)` (0.8) para `gato(H)`.
- Mover os pesos de `preto(L)` (1.5) e `fofinho(L)` (1.4) para o destino semanticamente mais próximo, que é `gato(H)`.
- Mover os pesos de `sentou-se(L)` (1.0) e `confortavelmente(L)` (1.3) para `sentou(H)`.
- Mover o peso de `tapete(L)` (1.2) para `tapete(H)`.

Mover Score - Passo 4 - Encontrar menor custo

O MoverScore não nos dá a distância bruta. Em vez disso, ele a normaliza para criar uma pontuação de similaridade, geralmente entre 0 e 1

$$\text{MoverScore}(h, r) = 1 - \frac{\text{CustoTotal}(h, r)}{\sum_i w_{h_i} + \sum_j w_{r_j}}$$

Para o exemplo, **MoverScore =** 0.375

WMD e Mover Score

Nos dariam uma medida de distância semântica, mas como desvantagem temos a complexidade computacional

Baseadas em similaridade - para o nosso caso

Podemos usar a similaridade de cosseno para obter uma visão geral da similaridade semântica entre a resposta gerada pelo LLM e a resposta humana

Porém, essa abordagem tem uma limitação: ao condensar toda a resposta em um único vetor denso, uma resposta que esteja, por exemplo:

- 80% correta
- 20% com erros graves

Poderia receber uma pontuação média de similaridade, que não reflete a gravidade dos erros, pois eles ficam menos perceptíveis na representação geral

LLM-as-a-judge

Refere-se ao uso de um LLM para avaliar a saída de outro LLM, geralmente através de um *prompt* detalhado que pede uma nota e uma justificativa em texto

- BLEURT - não fornece explicação, somente a nota, porque é baseada em BERT
- COMET - focada em tradução, também oferece a nota somente

BLEURT

A BLEURT (*Bilingual Evaluation Understudy with Representations from Transformers*) age mais como um juiz humano, a partir de uma rede neural que foi treinada para dar uma **nota,** do que como as métricas anteriores que são baseadas em similaridade ou contagem de *n-gramas.* A “compressão” dos textos foi feita com BERT

Valor mínimo = sem limite (pode ser negativo), **valor máximo=** sem limite (pode ser positivo).
Quanto **maior**, melhor. $O(n^2)$

BLEURT

Para cada frase, devem-se inserir um *tokens* especiais:

- [CLS] (*Classification*): É um *token* especial que vai, ao final do processo, carregar o significado agregado da relação entre as duas frases
- [SEP] (*Separator*): É um *token* que simplesmente indica onde uma frase termina e a outra começa

[CLS] O gato sentou no tapete. [SEP] O gato preto e fofinho sentou-se confortavelmente sobre o tapete. [SEP]

BLEURT

O modelo, que já foi treinado em bilhões de textos, lê a sequência inteira. Ele usa seus mecanismos de atenção para entender o contexto de cada palavra

Então, processa toda essa informação e a condensa em um único vetor numérico, que fica armazenado no *token* [CLS] do início

Como se fosse um vetor como um resumo rico e denso de quão semanticamente próximas, distantes, contraditórias ou redundantes as duas frases são

BLEURT - Resumo

Passo	O que Acontece?
1. Formatação	Juntamos as duas frases com tokens especiais.
2. Processamento	O modelo BERT lê e entende a relação entre as frases.
3. Pontuação	Uma camada final converte o entendimento complexo em uma única nota.

COMET

A COMET é uma métrica também classificada como LLM-as-a-judge, porém, ela é focada em traduções. A “compressão” dos textos foi feita com o modelo RoBERTa. Ela precisa de:

- ❑ Texto de origem no idioma de origem
- ❑ Texto traduzido - gerado por LLM
- ❑ Texto de referência - humano

Valor mínimo = sem limite (pode ser negativo), **valor máximo=** sem limite (pode ser positivo).
Quanto **maior**, melhor. $O(n^2)$

BLEURT - para o nosso caso

- Foi feita pensando em replicar um julgamento humano
- Seria importante traduzir as frases para o inglês antes
- A desvantagem é a “caixa-preta” de como ela realmente funciona e seus dados de treinamento

Referências

BLEU. Papineni et al. (2002). Disponível em:
https://www.researchgate.net/publication/2588204_BLEU_a_Method_for_Automatic_Evaluation_of_Machine_Translation

NIST. Doddington (2002). Disponível em:

https://dl.acm.org/doi/pdf/10.5555/1289189.1289273

TER. Swartz (2006). Disponível em:

https://www.researchgate.net/publication/228668187_A_study_of_translation_edit_rate_with_targeted_human_annotation

Referências

ROUGE. Lin (2004). Disponível em:
https://www.researchgate.net/publication/224890821_ROUGE_A_Package_for_Automatic_Evaluation_of_summaries

Sentence BERT-Similarity. Reimers e Gurevych (2019). Disponível em:

https://www.researchgate.net/publication/336996965_Sentence-BERT_Sentence_Embeddings_using_Siamese_BERT-Networks

Universal Sentence Encoder (USE) Similarity. Cer et al. (2018). Disponível em:

https://aclanthology.org/D18-2029/

Referências

SimCSE. Gao, Yao e Chen (2021). Disponível em:

https://aclanthology.org/2021.emnlp-main.552/

BERT Score. Zhang et al. (2019). DIsponível em:

https://www.researchgate.net/publication/332590189_BERTScore_Evaluating_Text_Generation_with_BERT

BART Score. Yuan, Neugib e Liu (2021). Disponível em:

https://dl.acm.org/doi/10.5555/3540261.3542349

Referências

Word Mover's Distance (WMD). Kusner et al. (2015). Disponível em:

https://dl.acm.org/doi/10.5555/3045118.3045221

Mover Score. Zhao et al. (2019). Disponível em:

https://www.researchgate.net/publication/335689825_MoverScore_Text_Generation_Evaluating_with_Contextualized_Embeddings_and_Earth_Mover_Distance

BLEURT. Sellan, Das e Parikh (2020). Disponível em:

https://research.google/pubs/bleurt-learning-robust-metrics-for-text-generation/

COMET. Rei et al. (2020) Disponível em:

https://aclanthology.org/2020.emnlp-main.213/?utm_source=chatgpt.com